

Explain Backtracking Example

Backtracking is a problem-solving technique where we try a choice, continue with it, and if it leads to a wrong/invalid solution, we undo that choice and try another one.

The easiest way to understand it is:

Choose → Explore → If wrong, Undo → Try another choice

Example: Generate all subsets

Suppose we have:

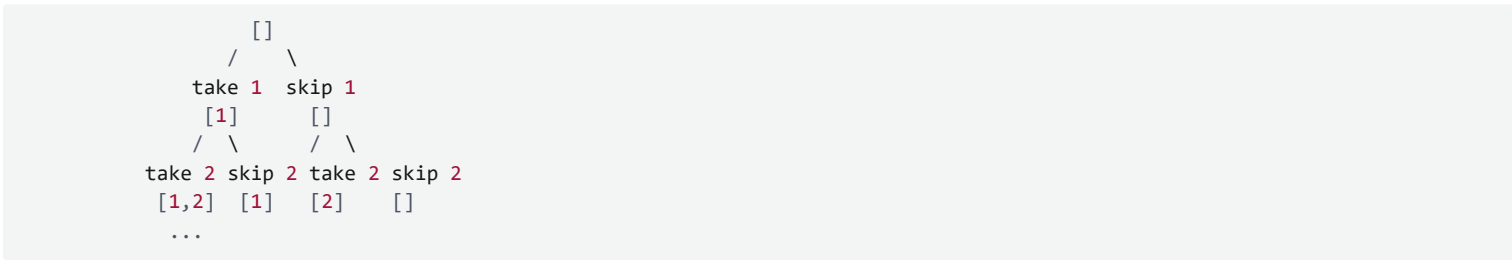
[1, 2, 3]

We want to generate all possible subsets.

At every number, we have 2 choices:

- 1. Take the number
- 2. Don't take the number

The recursion tree looks like:



Here is where **backtracking** happens.

```
void solve(vector<int>& nums, int index, vector<int>& current) {

    if (index == nums.size()) {
        for (int x : current)
            cout << x << " ";
        cout << endl;
        return;
    }

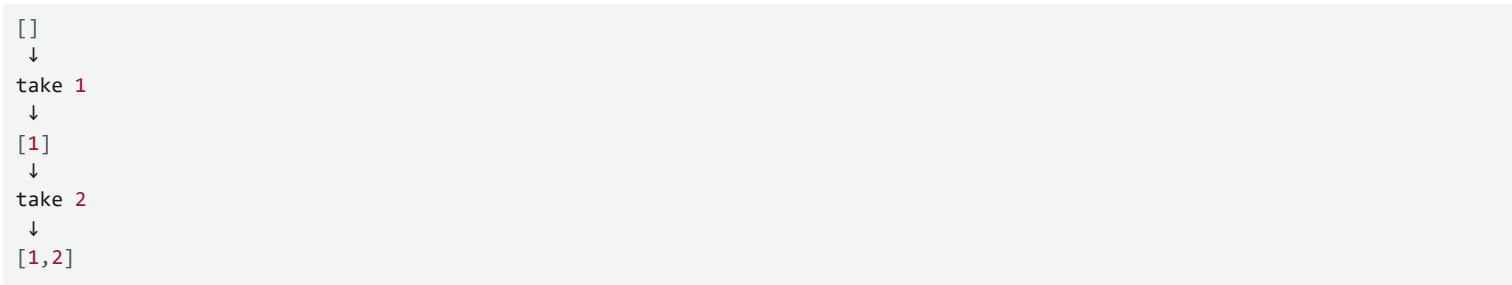
    // Take nums[index]
    current.push_back(nums[index]);
    solve(nums, index + 1, current);

    // BACKTRACK: undo the choice
    current.pop_back();

    // Don't take nums[index]
    solve(nums, index + 1, current);
}
```

Let's understand pop_back()

Suppose we are here:



Now we finish exploring [1,2].

We need to go back and try:

[1,3]

So we **undo** 2:

```
current.pop_back();
```

Now:

```
[1,2]
  ↓
remove 2
  ↓
[1]
```

Then we can try another choice:

```
[1,3]
```

That's **backtracking**.

A more practical example: Maze

Imagine a maze:

```
S . . #
# . . #
# . . E
```

You start at S.

You try going right:

```
S → . → .
```

If you reach a dead end:

```
S → . → . → #
```

you **go back** to the previous position and try another direction.

Conceptually:

```
try current path

if path is valid:
    continue

if path doesn't work:
    undo the move
    try another direction
```

Backtracking vs Normal Recursion

This distinction is important.

Normal recursion

```
Do something
  ↓
recursive call
  ↓
recursive call
  ↓
return
```

Backtracking

```
Make a choice
  ↓
recursive call
  ↓
does this choice work?
  /      \
yes       no
  ↓       ↓
continue UNDO choice
          ↓
        try another choice
```

So backtracking is basically recursion + choices + undoing choices.

Common Backtracking Problems

Once you understand this pattern, you'll see it in:

- Subsets
- Permutations
- Combination Sum
- N-Queens
- Sudoku
- Rat in a Maze
- Word Search
- Generate Parentheses

A very useful template to remember is:

```
void backtrack(...) {  
  
    if (answer_found) {  
        return;  
    }  
  
    for (each possible choice) {  
  
        // choose  
        make_choice();  
  
        // explore  
        backtrack(...);  
  
        // undo  
        undo_choice();  
    }  
}
```

The most important line to remember is:

```
choose → recurse → undo
```

That's the heart of backtracking.

Yes. There are some **strong signals** that tell you a problem is probably a backtracking problem.

The main idea is:

If the problem asks you to explore multiple choices, and after making one choice you may need to undo it and try another, think BACKTRACKING.

How to identify it

Look for these signals in the question:

1. "Find all..."

- Find all permutations
- Find all combinations
- Find all subsets
- Find all possible paths

2. At every step, you have multiple choices.

For example, with `[1,2,3]`, while generating subsets:

```
For 1:  
    take 1  
    OR  
    don't take 1  
  
For 2:  
    take 2  
    OR  
    don't take 2
```

This creates a **decision tree**:



When you see this kind of decision tree, recursion/backtracking should come to mind.

3. You need to undo a previous choice.

This is probably the biggest signal.

For example:

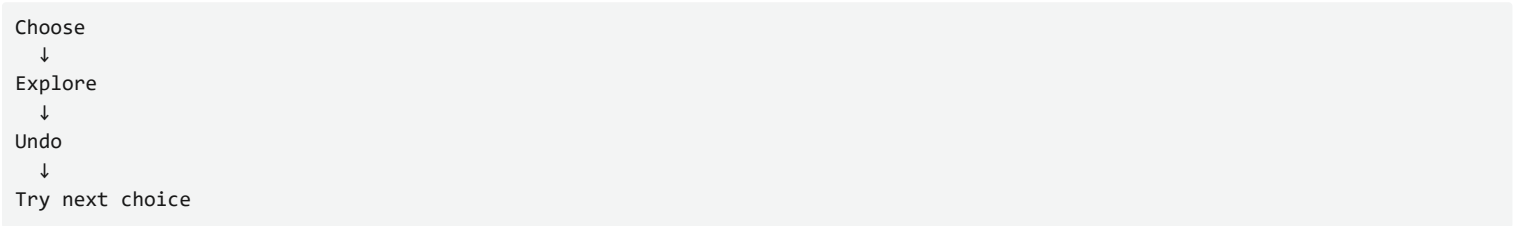
```

path.push_back(nums[i]); // CHOOSE

solve(...);              // EXPLORE

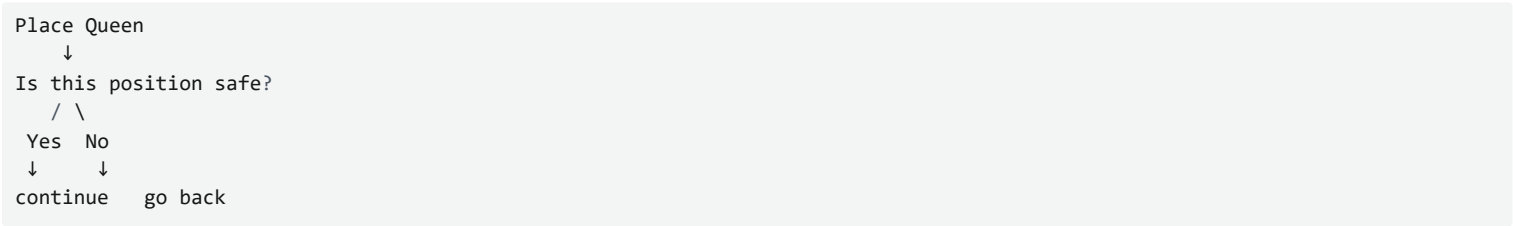
path.pop_back();          // UNDO
  
```

Think:

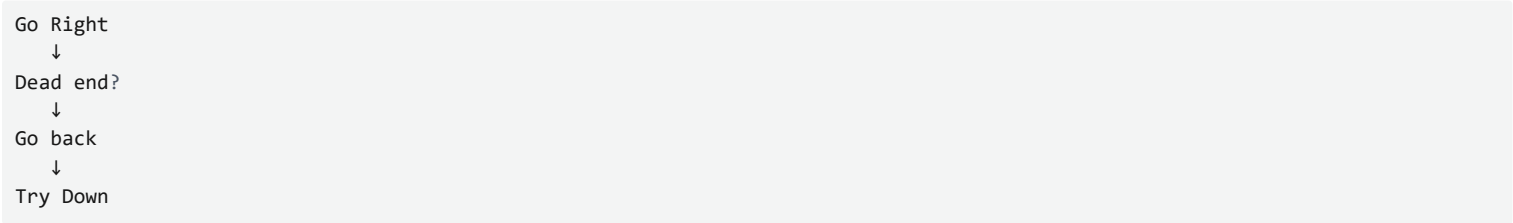


4. The problem has constraints that can make the current path invalid.

For example, in N-Queens:



Or in a maze:



This is classic backtracking.

A quick test

When reading a problem, ask yourself:

"If I make one choice now, do I need to explore what happens after that choice, then come back and try another choice?"

If yes, backtracking is a strong candidate.

For example, LeetCode 46 — Permutations:

```

Input:  [1,2,3]

Output:
[1,2,3]
[1,3,2]
[2,1,3]
[2,3,1]
[3,1,2]
[3,2,1]
  
```

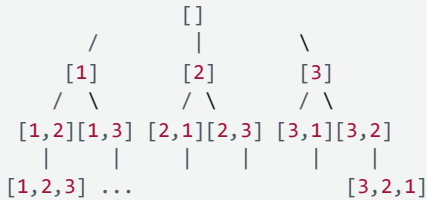
For the first position:

Choose 1
↓
Choose 2 or 3

Choose 2
↓
Choose 1 or 3

Choose 3
↓
Choose 1 or 2

So we're exploring choices:



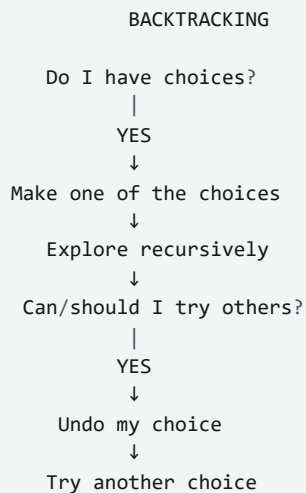
After generating [1,2,3], we need to **go back**, remove 3, then remove 2, and try 3 instead:

```
current.push_back(nums[i]); // choose

backtrack(...);           // explore

current.pop_back();        // undo
```

So the mental formula I recommend remembering is:



And the code pattern you'll see again and again is:

```
void solve(...) {

    if (base_case) {
        // save/print answer
        return;
    }

    for (each choice) {

        // 1. CHOOSE
        makeChoice();

        // 2. EXPLORE
        solve(...);

        // 3. UNDO
        undoChoice();
    }
}
```

So whenever you see **permutations, combinations, subsets, all paths, arranging things, placing things under constraints, Sudoku/N-Queens/maze**, ask yourself:

"Can I draw this problem as a tree of choices?"

If yes, backtracking is often the natural approach.

If you have to **explain Backtracking in class**, don't start with code. Start with the simple idea, then give an example, then show the code pattern.

You can explain it like this:



Simple classroom explanation

Backtracking — Classroom Explanation

Backtracking is a problem-solving technique where we try different possible choices one by one. If a choice does not lead to a valid solution, we undo that choice and try another one.

In simple words, the basic process is:

Choose → Explore → Undo → Try another choice

For example, suppose we have three numbers:

[1, 2, 3]

and we want to generate all possible subsets.

For every number, we have two choices:

- Take the number
- Don't take the number

For 1, we can take it:

[] → [1]

Then we explore what we can do with 2 and 3.

After finishing all possibilities starting with [1], we **undo our choice** and go back to:

[]

Then we explore the case where we don't take 1.

This process of making a choice, exploring it, and then undoing it is called **backtracking**.

A common backtracking structure is:

```
void backtrack() {  
  
    if (solution_found) {  
        return;  
    }  
  
    for (each possible choice) {  
  
        // Choose  
        makeChoice();  
  
        // Explore  
        backtrack();  
  
        // Undo  
        undoChoice();  
    }  
}
```

So, whenever a problem has **multiple choices at each step** and we need to explore different possibilities, backtracking can be useful.

Some common examples are:

- Permutations
- Subsets
- Combinations
- N-Queens
- Sudoku
- Maze problems

The easiest way to remember backtracking is:

"Try a choice → go deeper → if needed, come back → undo → try the next choice."

If you need to explain it in just 30 seconds

Say:

"Backtracking is a technique used to explore different possible solutions. At each step, we make a choice and recursively explore it. If that choice doesn't work or after finishing its possibilities, we undo the choice and try another one. So the basic pattern is Choose, Explore, Undo. Problems like permutations, subsets, Sudoku, N-Queens, and maze solving commonly use backtracking."

That's a good basic explanation for a classroom or viva because it explains *what it is, how it works, when to use it, and examples* without getting unnecessarily complicated.

The major difference is:

Recursion is a way of calling a function again and again.
Backtracking is a problem-solving technique that often uses recursion to try choices and undo them.

Simple comparison

Recursion	Backtracking
A programming technique	A problem-solving technique
Function calls itself	Usually uses recursion
Doesn't necessarily involve choices	Involves multiple choices
Doesn't necessarily undo anything	Usually makes and undoes choices
Used for trees, factorial, Fibonacci, etc.	Used for permutations, Sudoku, N-Queens, subsets, maze, etc.

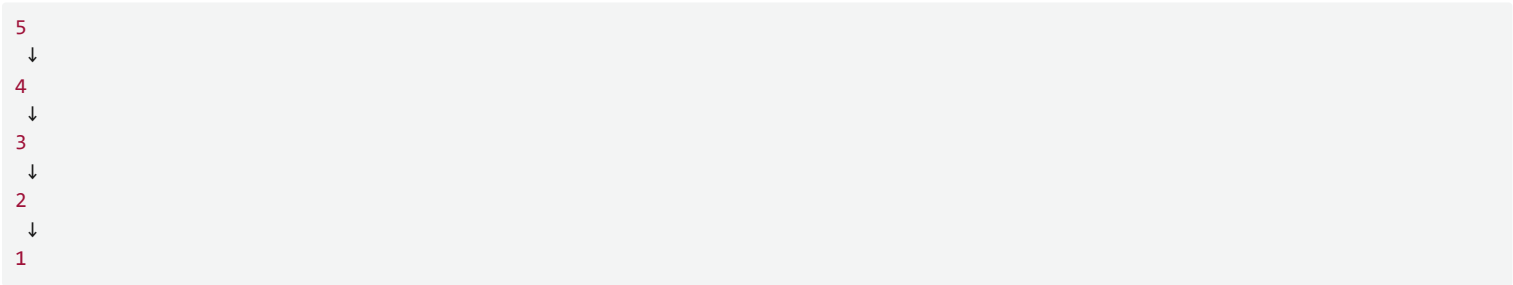
Example of recursion

Factorial:

```
int fact(int n) {
    if (n == 1)
        return 1;

    return n * fact(n - 1);
}
```

Here:



We're simply going deeper and then returning.

There is no choice and no undoing of a choice.

Example of backtracking

Suppose we want permutations of:

```
[1, 2, 3]
```

We choose 1:

```
[1]
```

Then choose 2:

```
[1,2]
```

Then choose 3:

```
[1,2,3]
```

Now we undo 3:

```
[1,2]
```

Then we try another choice.

Eventually:

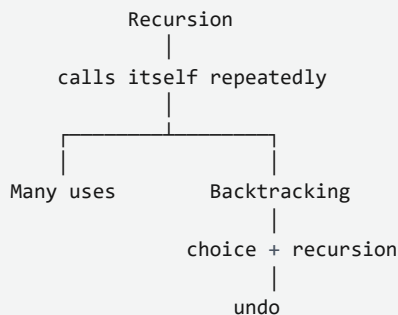
```
[1,2,3]
[1,3,2]
[2,1,3]
[2,3,1]
[3,1,2]
[3,2,1]
```

The important part is:

```
choose();
recursiveCall();
undo();
```

The key relationship

Think of it this way:



So:

Every backtracking solution commonly uses recursion, but every recursive solution is NOT backtracking.

A very easy interview/viva answer is:

"Recursion is the mechanism of solving a problem by calling the same function on a smaller subproblem, whereas backtracking is a strategy where we recursively explore different choices and undo a choice when we need to try another possibility."

Yes. If your teacher wants to test whether you **actually understand** recursion vs backtracking, they may ask questions that sound similar but test the subtle difference.



Tricky questions you can expect

1. Is every backtracking algorithm recursive?

A good answer:

Not necessarily. Backtracking is a problem-solving strategy, and recursion is the most common way to implement it. Backtracking can theoretically be implemented iteratively using an explicit stack.

So don't say "**backtracking = recursion.**"

2. Is every recursive algorithm a backtracking algorithm?

No. Recursion only means a function calls itself. Backtracking requires exploring choices and usually undoing a previous choice.

Example:

```
factorial(n)
```

is recursion, but not backtracking.

3. If I use `pop_back()`, does that automatically mean it's backtracking?

No. `pop_back()` alone doesn't define backtracking. Backtracking is about **undoing a decision so that another possible decision can be explored.**

4. Can backtracking exist without recursion?

Yes. We can manually maintain a stack and simulate the recursive process.

This is a slightly advanced question.

5. What is the "back" in backtracking?

The **back** means going back to a previous decision/state.

For example:

```
Choose A
↓
Choose B
↓
B doesn't work
↓
GO BACK
↓
Undo B
↓
Try C
```

6. Is DFS the same as backtracking?

This is a very common tricky one.

No. DFS is a traversal technique. Backtracking is a search/problem-solving technique that explores choices and may undo them.

However, backtracking is **often implemented using DFS**.

For example:

```
DFS:
Visit → Visit → Visit

Backtracking:
Choose → Explore → Undo → Try another
```

7. Is recursion required for backtracking?

No. Recursion is not the definition of backtracking. It is just a convenient implementation method.

8. Why can't we just use recursion instead of calling it backtracking?

Because recursion describes **how the function executes**, while backtracking describes **the strategy being used to search through possible solutions**.

This is an important conceptual distinction.

🔥 The most tricky one

Your teacher might ask:

"If backtracking uses recursion, then what's the difference? Aren't they the same thing?"

Answer:

No. Recursion is the mechanism, while backtracking is the strategy. Recursion tells us how we repeatedly solve smaller problems. Backtracking tells us how we explore different choices, and when necessary, undo a choice and try another.

Example:

```
Factorial
→ Recursion
→ No choices
→ No undo
→ Not backtracking

N-Queens
→ Recursion
→ Multiple choices
→ Explore each choice
→ Undo placement
→ Backtracking
```

🗨 One line to remember for your viva

"Recursion is about repeated function calls; backtracking is about exploring choices and reversing decisions."

And an even better mental model:

Recursion = HOW
Backtracking = WHAT STRATEGY

That's the distinction that usually separates a basic answer from a strong one.

No – **normal recursive Fibonacci is not considered backtracking**. This is a very good tricky point.

Consider:

```
int fib(int n) {  
    if (n <= 1)  
        return n;  
  
    return fib(n - 1) + fib(n - 2);  
}
```

You might think:

```
fib(n)  
├─ fib(n-1) ← choice 1  
└─ fib(n-2) ← choice 2
```

"There are two choices, so isn't this backtracking?"

No. The two calls are not alternative choices where we try one, undo it, and then try another. **Both subproblems are required** to calculate the answer.

The key difference

In Fibonacci:

```
fib(n)  
↓  
calculate fib(n-1)  
+  
calculate fib(n-2)  
↓  
combine both results
```

We're saying:

"I need BOTH answers."

There is no:

```
choose  
↓  
try  
↓  
undo  
↓  
try another
```

Compare with N-Queens

Suppose we're placing a queen:

```
Try column 1  
↓  
doesn't work  
↓  
REMOVE queen ← undo  
↓  
Try column 2  
↓  
...
```

Here we're saying:

"I'll try this choice. If it doesn't lead to a solution, I'll undo it and try another."

That's backtracking.

 The easiest test

Ask:

Are the branches alternatives, or do I need all the branches?

Fibonacci:

```
fib(n-1) AND fib(n-2)
```

Both are needed → recursion/divide-and-conquer, not backtracking.

Backtracking:

```
Try A
OR
Try B
OR
Try C
```

We explore alternatives and undo choices → **backtracking**.

So your observation that Fibonacci has "different branches and return values" is correct, but **having multiple recursive branches alone does not make something backtracking**.

Multiple recursive calls ≠ backtracking.

Choice + explore + undo + try another = backtracking.

Exactly! **This is the subtle part**. Your confusion is actually very important.

In subsets, we also have:

```
take 1
OR
don't take 1
```

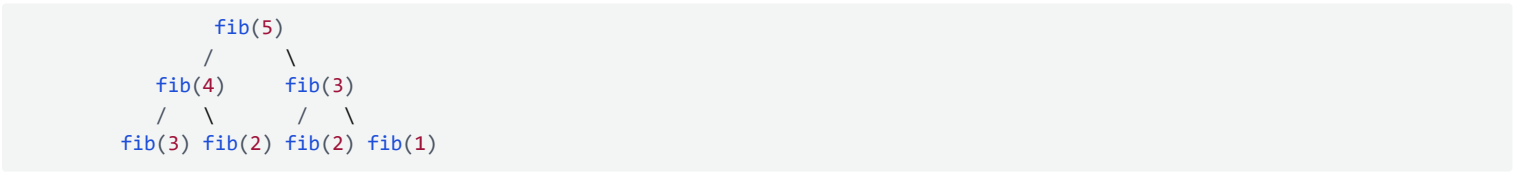
and we explore both. So why is that backtracking, while Fibonacci isn't?

The difference is **what the branches represent and whether we are maintaining a state/decision that gets undone**.

1. Fibonacci

```
fib(n - 1) + fib(n - 2);
```

Here the two branches are **two mathematical subproblems**.



We don't make a decision like:

"I'll choose fib(4) and if it fails, I'll remove it and try fib(3) ."

Instead:

"I need the answer of fib(4) AND the answer of fib(3) ."

There is **no changing shared solution state**.

2. Subsets

Suppose:

```
nums = [1,2]
```

We maintain a current subset:

```
current = []
```

We decide:

Take **1**

So:

```
current = [1]
```

Then we explore:

```
[1,2]  
[1]
```

Now we need to go back to the state before taking **1**.

So:

```
current.pop_back();
```

Now:

```
current = []
```

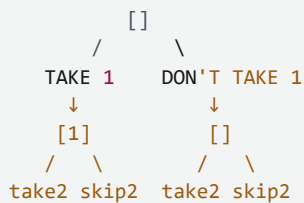
Then explore the other decision:

```
don't take 1
```

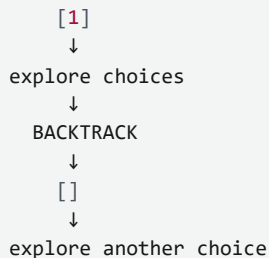
giving:

```
[2]  
[]
```

So the process is:



The important thing is:



Here's the real distinction

Don't define backtracking as simply:

"There are multiple recursive calls."

That's **too broad**.

Instead think:

Are we exploring a search space by making decisions on a current state and then reversing those decisions to explore another branch?

If yes → **backtracking**.

Why Fibonacci isn't backtracking

Look at what happens after:

```
fib(n - 1)
```

returns.

We don't do:

```
undo fib(n - 1);
```

We simply use its returned value:

```
return fib(n - 1) + fib(n - 2);
```

The previous computation isn't a **choice/state that we're undoing**.

But subsets have:

```
current.push_back(nums[i]); // make decision
solve(...);                // explore
current.pop_back();         // undo decision
```

That push → recurse → pop pattern is the classic backtracking pattern.



Think about it like this

Imagine you're in a building.

Fibonacci:

```
"I need information from Room A and Room B."
```

So you visit both.

Backtracking:

```
"I can take Door A or Door B. Let me try A. If it doesn't give me what I want, I'll come back, undo my decision, and try B."
```

That's the fundamental difference.

One important clarification

Backtracking **does not mean we only keep the successful choice**.

We can absolutely explore **both/all alternatives**, as in subsets.

The defining idea is:

Make a decision → explore that decision → restore the previous state → explore another decision.

So:

```
Fibonacci:
branch → branch → combine results
      ✖ no decision undo

Subsets:
choose → explore → undo → choose another
      ✔ backtracking
```

This is why **subsets are a classic backtracking problem**, even though every branch is ultimately included in the answer.

Backtracking — Complete Summary

The easiest way to understand the whole concept is:

```
Recursion is a technique for executing a problem. Backtracking is a strategy for searching through different possibilities.
```

1. What is recursion?

Recursion means a function calls itself to solve a smaller or simpler version of the same problem.

Example: Fibonacci

```
int fib(int n) {
    if (n <= 1)
        return n;

    return fib(n - 1) + fib(n - 2);
}
```

Here:

```

fib(5)
├── fib(4)
│   ├── fib(3)
│   └── fib(2)
└── fib(3)
    ├── fib(2)
    └── fib(1)

```

There are multiple branches, but **this doesn't make it backtracking**.

Why?

Because we need the results of **both**:

```
fib(n-1) AND fib(n-2)
```

We aren't making a decision, modifying a solution, undoing it, and trying another possibility.

2. What is backtracking?

Backtracking is a technique where we:

1. **Make** a choice
2. **Explore** that choice
3. **Undo** the choice
4. **Try** another choice

The most important formula:

```
Choose → Explore → Undo → Try another
```

For example, generating subsets of:

```
[1, 2, 3]
```

For every element we have choices:

```

Take 1
OR
Don't take 1

```

We explore one possibility:

```

[]
↓
[1]
↓
[1,2]
↓
[1,2,3]

```

Then we **undo** the last decision:

```

[1,2,3]
↓
remove 3
↓
[1,2]

```

Then try another possibility.

In code:

```

current.push_back(nums[i]); // CHOOSE

backtrack(...);           // EXPLORE

current.pop_back();        // UNDO

```

That push → recurse → pop pattern is one of the strongest signs of backtracking.

3. Why Fibonacci is NOT backtracking

This was the tricky part we discussed.

Fibonacci has:

```
fib(n)
  /   \
n-1   n-2
```

You might think:

"There are two choices, so it must be backtracking."

But these aren't **alternative choices**.

We need:

```
fib(n-1) + fib(n-2)
```

Both branches are required.

There is no:

```
choose
↓
explore
↓
undo
↓
try another
```

Instead:

```
solve subproblem A
+
solve subproblem B
↓
combine their answers
```

So:

Multiple recursive calls do NOT automatically mean backtracking.

4. Why subsets ARE backtracking

For subsets:

```
    []
   / \
take 1 skip 1
  ↓   ↓
[1]  []
```

These represent **different decisions**.

We choose one path, explore it, then restore the previous state:

```
current = []

take 1
current = [1]

explore...

undo 1
current = []

try skip 1
```

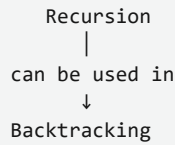
So we're searching through a **decision tree**.

5. Recursion vs Backtracking

Recursion	Backtracking
Programming technique	Problem-solving/search strategy
Function calls itself	Usually uses recursion
May have one or multiple recursive calls	Usually has multiple possible choices
Doesn't require choices	Based on choices/possibilities

Recursion	Backtracking
Doesn't require undoing state	Usually involves undoing a choice
Example: factorial, Fibonacci, tree traversal	Example: subsets, permutations, Sudoku, N-Queens
Focus: recursive structure	Focus: exploring possibilities

The most important relationship:



But:

Every recursion $\neq$ backtracking

6. The biggest difference

Think about the question:

"What am I doing after I return from the recursive call?"

Normal recursion

You usually **use the returned result**.

```
return fib(n-1) + fib(n-2);
```

The recursive calls produce values.

Backtracking

You often **restore/change the state** and try another possibility.

```
path.push_back(x);

backtrack();

path.pop_back();
```

The recursive call explores a possibility, and afterward we **undo the decision**.

7. How to identify a backtracking problem

When you read a question, ask these questions:

Question 1:

Are there multiple possible choices at each step?

For example:

```
Take / Don't Take
Left / Right
Choose 1 / Choose 2 / Choose 3
```

Question 2:

Do I need to explore different possibilities?

For example:

```
all subsets
all permutations
all valid combinations
all possible paths
```

Question 3:

Do I need to undo my previous choice before trying another?

If yes, you should strongly think about **backtracking**.

8. Common backtracking problems

You will frequently see it in:

Subsets
Permutations
Combinations
Combination Sum
N-Queens
Sudoku
Rat in a Maze
Word Search
Generate Parentheses

Most have this structure:

```
void backtrack(...) {  
  
    if (base_case) {  
        // save answer  
        return;  
    }  
  
    for (each choice) {  
  
        // 1. CHOOSE  
        makeChoice();  
  
        // 2. EXPLORE  
        backtrack(...);  
  
        // 3. UNDO  
        undoChoice();  
    }  
}
```



Final mental model

Remember these three levels:

Recursion

Function
↓
calls itself
↓
smaller problem

Backtracking

Make a choice
↓
Explore recursively
↓
Undo the choice
↓
Try another choice

Fibonacci

Solve A
+
Solve B
↓
Combine

Not backtracking.

Subsets

Take
↓
Explore
↓
Undo
↓
Don't take
↓
Explore

Backtracking.

 Best one-line answer for an interview/viva

"Recursion is a technique where a function solves a problem by calling itself, while backtracking is a search strategy that uses recursion or another mechanism to explore different choices, undo a previous choice, and try another possibility."

And the phrase you should remember forever:

Backtracking = Choice + Exploration + Undoing the choice.